

Method	Input	Iteration	Idea Behind Method	Required for Convergence	Pros	Cons
Bisection	$f(x)$, endpoints (a, b), tolerance	Find the center of (a, b). Find $f(c)$. Pick new interval (a, c) or (c, b) accordingly. Repeat until error < tolerance	Know there is a root in (a, b). Cut interval in half until center of the interval is approximately the root. $f(\text{center}) = 0$	$f(x)$ is continuous on [a, b], $f(a) \cdot f(b) < 0$	Simple, guaranteed to converge with appropriate (a,b)	It is super inefficient. (slow linear convergence)
Fixed Point	$f(x)$, X_0 , tolerance, $N_{\max}$ iterations, $g(x) = x - f(x)$ We are finding the root of g	$g(x) = x - f(x)$ Find $g(X_0)$. $X_1 = g(X_0)$... Find $g(X_{n-1})$. $X_n = f(X_{n-1})$. Repeat until $ X_n - X_{n-1} < \text{tol}$	Finding intersection of $y = x$ and $g(x)$. Fixed point of $f(x)$ is the root of $g(x)$	$g(x) = x - f(x)$ $ g'(p) < k$ where $0 < k < 1$	Simple, typically faster than bisection, no derivatives	Slow linear convergence, easily diverges
Newton	$f(x)$, $f'(x)$, X_0 , tolerance, $N_{\max}$ iterations	$X = X_0 - f(X_0)/f'(X_0)$... $X_n = X_{n-1} - f(X_{n-1})/f'(X_{n-1})$ Repeat until relative error < tolerance	Keep finding root of tangent lines of $f(x)$ until root of tangent line is approximately root of $f(x)$	$f'(X_0)$ is not 0. X_0 sufficiently close to root. $f(x)$ is C^2	Fast quadratic convergence	$f(x)$ must have continuous first and second derivatives in the interval, must calculate derivatives by hand
Secant	X_0, X_1 , $f(x)$, $N_{\max}$, tolerance,	$X_{n+1} = X_n - [(X_n - X_{n-1}) / (f(x_n) - f(x_{n-1}))]$ This is the root of a secant line	Keep finding root of secant line until root of secant is approximately root of $f(x)$	Sufficiently close initial guesses	Superlinear convergence, doesn't need derivatives	Needs two initial x guesses.

3.1 EXERCISES

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = g(x_n)$$

$$g'(x_n) = - \left(\frac{f'(x_n)^2 - f(x_n)f''(x_n)}{f'(x_n)^2} \right)$$

$$= -1 + \frac{f(x_n)f''(x_n)}{f'(x_n)^2}$$

$$|g'(x_0^*)| < 1$$

- 1)
- 2) I replaced the size of error check with if d is within the tolerance using the formula above.
- 3) I did need to change the input of the original bisection. We no longer need tolerance since we are not using a tolerance to check when our approximation is close enough. We do need to add the first and second derivative of $f(x)$ to check if our approximation is in the basin of convergence.
- 4) I added the Newton code to the end of the bisection code and made astar the input for the Newton algorithm. I had to add tolerance and Nmax into the inputs for the Newton section. I also altered what Newton returns so it matches what bisection error checks return.
- 5) The advantage is that the bisection part of the code guarantees that we are starting Newton's method within the basin of convergence so the function is guaranteed to converge. The limitations of this new method is that we are still running bisection so the function is still linearly convergent (inefficient), to find an x value within the basin of convergence. Another disadvantage is that we need the first and second derivative of the function.
- 6) **Lab ended when I was implementing my code and fixing errors.**